Лабораторная работа №4. Параллельное умножение матриц с использованием OpenMP

1. Реализованные варианты и среда измерений

Реализованы пять вариантов: `ijk_seq` (базовый последовательный), `ijk_omp` (параллельный по i), `ikj_seq` (последовательный с порядком i, k, j), `ikj_omp` (параллельный по i) и `blocked_omp` (блочный параллельный). Измерения проводились на компьютере Apple M2 (8 физических/логических ядер) с компилятором gcc-16 (Homebrew GCC 16.2.0), ключи `-O2 -fopenmp`. Время замерялось через `omp_get_wtime()` как медиана 5 запусков после прогрева. Во всех запусках проверка корректности прошла успешно (`maxdiff=0.00e+00`).

2. Время выполнения для разных размеров матриц

Зафиксировано: число потоков = 8, размер блока = 32.

N	ijk_seq, с	ijk_omp, с	ikj_seq, с	ikj_omp, с	blocked_omp, с
128	0.00161	0.00044	0.00079	0.00029	0.00027
256	0.01587	0.00344	0.00600	0.00140	0.00148
512	0.14042	0.02825	0.04265	0.00919	0.01016
1024	1.20984	0.32041	0.32960	0.06867	0.08132

3. Время, ускорение и эффективность от числа потоков

N = 1024, размер блока = 32. Ускорение $S = T_{seq} / T_p$, эффективность $E = S / p$. $T_{ijk_seq} = 1.20984$ с, $T_{ikj_seq} = 0.32960$ с.

Потоки	ijk_omp			ikj_omp			blocked_omp, с
	Время, с	S	E	Время, с	S	E	
1	1.20972	1.00	1.00	0.33133	0.99	0.99	0.40156
2	0.72019	1.68	0.84	0.17125	1.92	0.96	0.23040
4	0.36434	3.32	0.83	0.09906	3.33	0.83	0.12687
8	0.31723	3.81	0.48	0.07032	4.69	0.59	0.08925

4. Сравнение ijk и ikj, влияние размера блока

Вариант `ikj` быстрее `ijk` уже в последовательном исполнении: при N = 1024 время `ikj_seq` составляет 0.33 с против 1.21 с у `ijk_seq` (разница в 3.6 раза). В параллельном исполнении `ikj_omp` также стабильно быстрее `ijk_omp` при любом числе потоков. Влияние размера блока при N = 1024 и 8 потоках:

Размер блока b	Время blocked_omp, с	Отношение $T_{ijk_seq} / T_{blocked_omp}$
16	0.09750	12.41
32	0.08713	13.88
64	0.07991	15.14

Наилучший результат показывает блок 64×64: время 0.07991 с, что в 15.14 раза быстрее последовательного `ijk_seq`.

5. Вывод

Быстрее всех оказался вариант `ikj_omp` и блочный `blocked_omp` (с блоком 64). Это объясняется локальностью памяти: в `ikj` внутренний цикл по j обеспечивает последовательный доступ к элементам матриц B и C, что позволяет эффективно использовать кэш-линии и аппаратный префетчер, тогда как в `ijk` доступ к B идёт с шагом N, вызывая постоянные промахи кэша. Блочный алгоритм дополнительно гарантирует, что подматрица нужного размера (64×64 double = 32 КБ) целиком помещается в кэш L1 (64 КБ), минимизируя обмены с оперативной памятью.

Ускорение перестаёт расти при переходе от 4 к 8 потокам: для `ijk_omp` оно увеличивается лишь с 3.32 до 3.81, а эффективность падает до 0.48; для `ikj_omp` — с 3.33 до 4.69 (E = 0.59). Основные ограничения: (1) задача является memory-bound — при N = 1024 матрицы занимают по 8 МБ, что превышает кэш L2 (4 МБ), и потоки начинают конкурировать за общую пропускную способность памяти; (2) накладные расходы OpenMP на создание параллельной области и синхронизацию; (3) архитектурные ограничения Apple M2 — при 8 потоках шина памяти насыщается, и добавление новых потоков уже не даёт прироста.